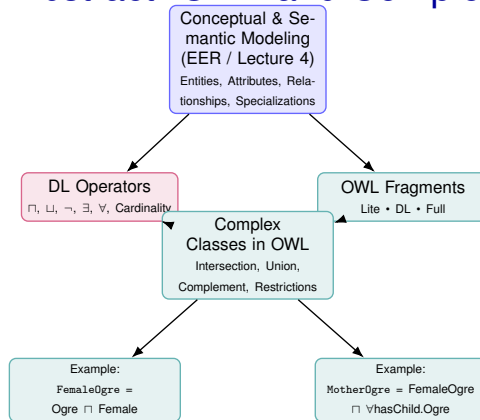


OWL and Complex Classes (Lecture 8)



Graphical Abstract: OWL and Complex Classes



Summary: OWL builds on conceptual modeling and Description Logics to formally define complex classes, enabling reasoning, inference, and semantic interoperability.

Agenda

1

More Properties in OWL

Functional, inverse, transitive, symmetric, disjoint properties

2

Defining Complex Classes

Using DL operators: intersection, union, complement, restrictions

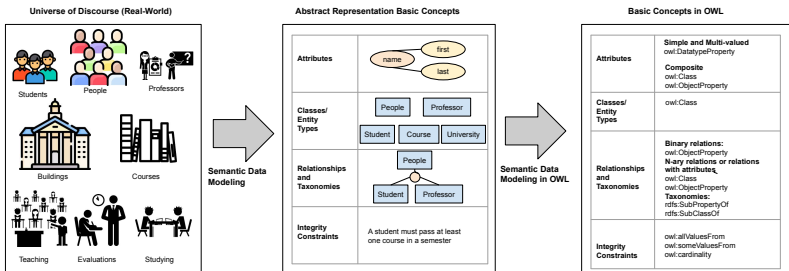
3

Modeling Basic Concepts in OWL

Attributes, relationships, taxonomies, entity structures

Semantic Data Modeling [1] (1/2) (From Lecture 4)

- **Definition:** Identifies and represents key concepts relevant to a universe of discourse.
- **Purpose:**
 - Capture and express **knowledge** that characterizes key concepts.
 - Enable representation in a **knowledge base**.



Semantic Data Modeling [1] (2/2) (From Lecture 4)

Basic Concepts:

- **Entities and Classes:** Abstract representations of real-world objects or concepts.
- **Attributes:** Properties describing the characteristics of entities.
- **Relationships:** Define associations between entities.
- **Taxonomies:**
 - Organize entities into **hierarchical structures**.
 - Support parent-child relationships for semantic classification.
- **Integrity Constraints:**
 - Ensure **consistency** in knowledge representation.
 - Include constraints like **type hierarchies** and **class disjointness**.

OWL Class Constructors (from Lecture 7)

Boolean Constructors

- **Intersection**
`owl:intersectionOf`
e.g. `Ogre` $\sqcap$ `Female`
- **Union**
`owl:unionOf`
e.g. `Father` $\sqcup$ `Mother`
- **Complement**
`owl:complementOf`
e.g. `NonOgre` $\equiv \neg$ `Ogre`

Class Axioms

- **Equivalence:**
`owl:equivalentClass`
- **SubClass:** `rdfs:subClassOf`
- **Enumeration:** `owl:oneOf` (e.g. `{CS, Math, Physics}`)

Restrictions

- **Existential** ($\exists$)
`owl:someValuesFrom`
“has at least one child who is an Ogre”
- **Universal** ($\forall$)
`owl:allValuesFrom`
“all children are Ogres”
- **Has-value**
`owl:hasValue`
e.g. `gender = ex:female`

OWL Property Operators and Cardinality

Property Characteristics

- **Functional / Inverse Functional**
`owl:FunctionalProperty`
`owl:InverseFunctionalProperty`
- **Transitive / Symmetric / Asymmetric**
- **Reflexive / Irreflexive**
- **Inverse** properties:
`owl:inverseOf`
- **Disjoint** properties:
`owl:propertyDisjointWith`

Cardinality

- **Unqualified**
`minCardinality`,
`maxCardinality`, `cardinality`
- **Qualified**
`minQualifiedCardinality`,
`maxQualifiedCardinality`,
`qualifiedCardinality`
- Example: “has at least one child who is a LivingBeing”

Identity and Disjointness

- Individuals: `owl:sameAs`,
`owl:differentFrom`,
`owl:AllDifferent`
- Classes: `owl:disjointWith`

OWL Fragments (1/2)

OWL is structured into three fragments to balance expressiveness and computational feasibility.

OWL Lite

- Simplest fragment
- Basic hierarchies
- Simple constraints

Example

```
ex:Parent rdfs:subClassOf
ex:Person.
ex:Person a owl:Class.
```

OWL DL

- Based on Description Logics
- Good balance of expressiveness and decidability

Example

```
ex:Parent
owl:equivalentClass [
  owl:intersectionOf (
    ex:Person
    [ owl:onProperty
ex:hasChild;
  owl:someValuesFrom
ex:Child ] ) ] .
```

OWL Full

- Maximum expressiveness
- Classes as individuals
- No guarantees for reasoning

Example

```
ex:ClassA rdfs:subClassOf
ex:ClassB.
ex:ClassA a ex:Instance.
```


OWL Properties for Expressing Inequality

OWL defines properties to express relationships and constraints:

- **owl:propertyDisjointWith:** Two properties cannot share pairs of individuals.

Example:

```
ex:hasParent owl:propertyDisjointWith ex:hasSpouse.
```

- **owl:disjointWith:** Two classes cannot share instances.

Example:

```
ex:Male owl:disjointWith ex:Female.  
ex:Shrek a ex:Male.
```

- **owl:differentFrom:** Two individuals are distinct.

Example:

```
ex:Shrek owl:differentFrom ex:Fiona.
```

- **owl:AllDifferent:** Declares all individuals in a list as distinct.

Example:

```
_:group a owl:AllDifferent;  
    owl:distinctMembers (ex:Shrek ex:Fiona ex:Donkey).
```

OWL Property Expressing Complement

OWL provides the ability to define classes as complements, which means the set of individuals that are not members of a specified class.

Definition:

`owl:complementOf`: Defines a class as the complement of another class.

Example:

```
ex:NonOgre owl:complementOf ex:Ogre.
```

```
ex:Fiona a ex:Ogre.  
ex:Donkey a ex:NonOgre.
```

Inferred:

```
ex:Shrek a ex:NonOgre => False.
```

Note:

`owl:complementOf` ensures strict negation, such that the two classes (e.g., `ex:Ogre` and `ex:NonOgre`) do not share any instances.

Introduction to Complex Classes in OWL

Why Do We Need Complex Classes?

- Simple classes (e.g., *Ogre*, *Female*) are often not enough to describe rich domain concepts.
- OWL allows us to **combine**, **restrict**, and **relate** classes using logical constructors.
- These combinations form **complex classes** that model more precise, semantically meaningful categories.

What Can We Express?

- **Intersection** — individuals must satisfy *all* component classes (e.g., a *MotherOgre* is an *Ogre* *and* *Female* *and* has *Ogre* children).
- **Union** — individuals satisfy *at least one* of several classes.
- **Restrictions** — constrain properties (e.g., “all children are *Ogres*”, “has at least one child who is an *Ogre*”).

Defining Complex Classes in OWL (1/2)

What is a Complex Class?

- A **complex class** is defined using logical expressions in OWL to combine, restrict, or intersect simpler classes and properties.
- It allows for the creation of sophisticated class definitions to represent intricate relationships and constraints in the domain.

Examples:

```
ex:FemaleOgre owl:equivalentClass
  [owl:intersectionOf (ex:Ogre ex:Female)].
ex:MotherOgre owl:equivalentClass
  [owl:intersectionOf
    (ex:FemaleOgre
      [owl:allValuesFrom ex:Ogre;
        owl:onProperty ex:hasChild])].
ex:C0 owl:equivalentClass
  [owl:unionOf (ex:C1 ex:C2 ex:C3 ex:C4 ex:C5)].

ex:C0 owl:subClassOf
  [owl:intersectionOf (ex:C1 ex:C2 ex:C3 ex:C4)].
```

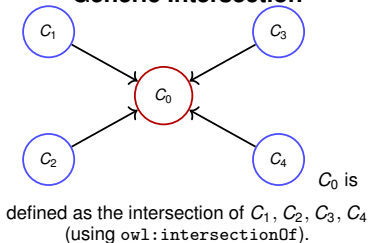
Defining Complex Classes in OWL (2/2)

Explanation:

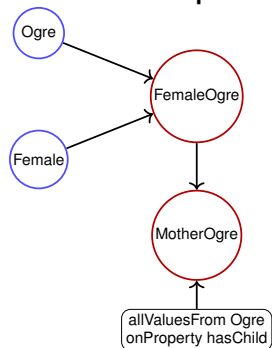
- **Intersection:** Combines multiple classes; an individual must belong to all the combined classes (e.g., `ex:MotherOgre` is both a `FemaleOgre` and has children that are all `ex:Ogres`).
- **Union:** Combines multiple classes; an individual must belong to at least one of the combined classes (e.g., `ex:C0` is the union of `ex:C1`, `ex:C2`, etc.).
- **Restrictions:** Adds constraints to a class definition, such as all values for a property being of a certain type.

Visualizing Intersection in OWL

Generic Intersection



Domain Examples



$\text{FemaleOgre} = \text{Ogre} \sqcap \text{Female}$
 $\text{MotherOgre} = \text{FemaleOgre} \sqcap \text{restriction}$

Idea: `owl:intersectionOf` combines several class expressions into one complex class that must satisfy all of them.

Examples of Representing Intersection

Definition: Suppose the class `ex:C0` is defined as the intersection of `ex:C1`, `ex:C2` upto `ex:Cn`, where $n > 1$. The operator `owl:intersectionOf` needs to be used.

Intersection Example

```
ex:C0 owl:equivalentClass [owl:intersectionOf (ex:C1 ex:C2 ex:C3 ex:C4)].
```

Note: $n = 4$ in this example.

Examples of Intersection

A female ogre is an ogre and a female.

Definition

```
ex:FemaleOgre owl:equivalentClass [owl:intersectionOf (ex:Ogre ex:Female)].
```

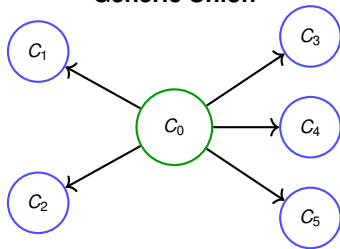
A mother ogre is a female ogre whose all her children are ogres.

Definition

```
ex:MotherOgre owl:equivalentClass [owl:intersectionOf (ex:FemaleOgre  
[owl:allValuesFrom ex:Ogre; owl:onProperty ex:hasChild])].
```

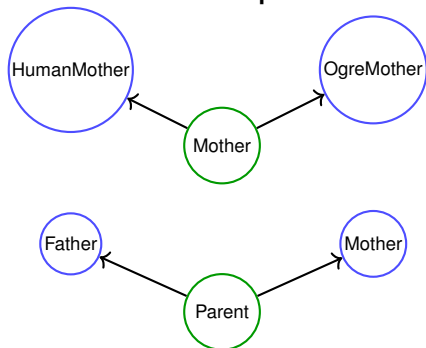

Visualizing Union in OWL

Generic Union



C_0 is defined as the union of C_1, C_2, C_3, C_4, C_5 (using `owl:unionOf`).

Domain Examples



$\text{Mother} = \text{HumanMother} \sqcup \text{OgreMother}$
 $\text{Parent} = \text{Father} \sqcup \text{Mother}$

Idea: `owl:unionOf` defines a class that contains all individuals belonging to *at least one* of the component classes.

Representing Union

Definition: Suppose the class `ex:C0` is defined as the union of `ex:C1`, `ex:C2` upto `ex:Cn`, where $n > 1$. The operator `owl:unionOf` needs to be used.

Union Example

```
ex:C0 owl:equivalentClass [owl:unionOf (ex:C1 ex:C2 ex:C3 ex:C4 ex:C5)].
```

Note: $n = 5$ in this example.

Examples of Union

A mother is either a human mother or an ogre mother.

Definition

```
ex:Mother owl:equivalentClass [owl:unionOf (ex:HumanMother ex:OgreMother)].
```

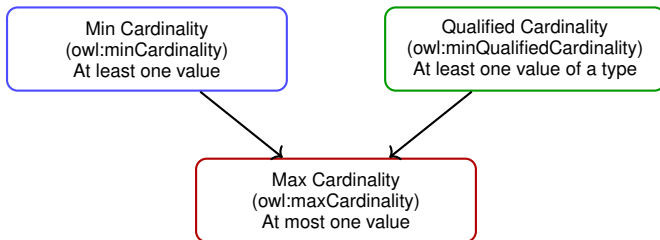
A parent is either a father or a mother.

Definition

```
ex:Parent owl:equivalentClass [owl:unionOf (ex:Father ex:Mother)].
```

Cardinality Constraints in OWL

OWL allows us to express how many values a property may have. Below are the three most common types of cardinality constraints:



Cardinality restrictions enable precise class definitions by limiting how often a property may be used and, optionally, what type of value it must have.

Representing Cardinality

A mother is a female who has at least one child.

Definition

```
ex:Mother owl:equivalentClass [owl:intersectionOf (ex:Female [owl:minCardinality  
"1"^^xsd:nonNegativeInteger; owl:onProperty ex:hasChild])].
```

Representing Qualified Cardinality

A mother is a female who has at least one child who is a living being.

Definition 1

```
ex:Mother owl:equivalentClass [owl:intersectionOf (ex:Female  
[owl:minQualifiedCardinality "1"^^xsd:nonNegativeInteger; owl:onProperty  
ex:hasChild; owl:onClass ex:LivingBeing])].
```

Another equivalent representation:

Definition 2

```
ex:Mother owl:equivalentClass [owl:intersectionOf (ex:Female [owl:someValuesFrom  
ex:LivingBeing; owl:onProperty ex:hasChild])].
```

Cardinality and Functional Properties

A person has at most one spouse.

- **Definition:**

```
ex:Person owl:equivalentClass  
  [owl:maxCardinality "1"^^xsd:nonNegativeInteger;  
    owl:onProperty ex:hasSpouse].
```

- **Equivalent Representation:**

```
ex:hasSpouse a rdf:Property, owl:FunctionalProperty;  
             rdfs:domain ex:Person.
```

Inverse Properties

A person is the child of his/her mother, and everyone has only one mother.

- **Property Definitions:**

```
ex:isMotherOf a rdf:Property;  
              rdfs:domain ex:Mother;  
              rdfs:range ex:Person;  
              rdfs:subPropertyOf ex:hasChild.
```

```
ex:isChildOfMother a rdf:Property, owl:InverseFunctionalProperty;  
                  owl:inverseOf ex:isMotherOf.
```

- **Class Definition:**

```
ex:Person owl:equivalentClass  
  [owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger;  
   owl:onProperty ex:isChildOfMother;  
   owl:onClass ex:Mother].
```


More about Properties

A daughter is a female descendant of her mother, and any female of whom her mother is a female descendant.

■ Property Definitions:

```
ex:femaleDescendant a rdf:Property, owl:TransitiveProperty,  
owl:IrreflexiveProperty;  
    rdfs:domain ex:Daughter;  
    rdfs:range ex:Mother.
```

```
ex:Mother owl:equivalentClass  
    [owl:intersectionOf (ex:Female  
        [owl:someValuesFrom ex:Person;  
        owl:onProperty ex:hasChild]])].
```

■ Class Definitions:

```
ex:Daughter owl:equivalentClass  
    [owl:someValuesFrom ex:Mother; owl:onProperty ex:femaleDescendant].
```

```
ex:Female owl:equivalentClass  
    [owl:intersectionOf (ex:Person [owl:hasValue ex:female;  
        owl:onProperty ex:gender]])].
```

Example (1/5)



```

ex:Felicia a ex:Person; ex:gender ex:female.
ex:HumanFiona a ex:Person; ex:gender ex:female.
ex:OgreFiona a ex:Person; ex:gender ex:female.
ex:QueenLilian a ex:Person; ex:gender ex:female.
ex:LiliansMother a ex:Person; ex:gender ex:female.
ex:Fergus a ex:Person; ex:gender ex:male.

ex:Felicia ex:isChildOfMother ex:HumanFiona, ex:OgreFiona.
ex:Fergus ex:isChildOfMother ex:HumanFiona, ex:OgreFiona.
ex:HumanFiona ex:isChildOfMother ex:QueenLilian .
ex:OgreFiona ex:isChildOfMother ex:QueenLilian .
ex:QueenLilian ex:isChildOfMother ex:LiliansMother .

ex:Felicia ex:femaleDescendant ex:HumanFiona, ex:OgreFiona.
ex:HumanFiona ex:femaleDescendant ex:QueenLilian .
ex:OgreFiona ex:femaleDescendant ex:QueenLilian .
ex:QueenLilian ex:femaleDescendant ex:LiliansMother .
    
```

Inference from Complex Class Definition (2/5)

Definition of Complex Class:

```
ex:Female owl:equivalentClass
  [owl:intersectionOf (
    ex:Person
    [owl:hasValue ex:female;
     owl:onProperty ex:gender]])].
```

Consider the RDF triples presented in slide 26

Inferred Triples

```
ex:Felicia a ex:Female.
ex:HumanFiona a ex:Female.
ex:OgreFiona a ex:Female.
ex:QueenLilian a ex:Female.
ex:LiliansMother a ex:Female.
```

Inference from Complex Class Definition (3/5)

Definition of Complex Class:

```
ex:Mother owl:equivalentClass  
  [owl:intersectionOf (  
    ex:Female  
    [owl:someValuesFrom ex:Person;  
      owl:onProperty ex:hasChild]])].
```

Consider the RDF triples presented in slide 26.

Inferred Triples

```
ex:HumanFiona a ex:Mother.  
ex:OgreFiona a ex:Mother.  
ex:QueenLilian a ex:Mother.  
ex:LiliansMother a ex:Mother.
```

Note:

The derived triples were inferred based on the definition of the complex class `ex:Mother`. The properties `owl:someValuesFrom` and `owl:onProperty` specify that `ex:Mother` is a `Female` having at least one child who is a `Person` via the property `ex:hasChild`.

Inference from Complex Class Definition (4/5)

Definition of Complex Class:

```
ex:Daughter owl:equivalentClass
  [owl:someValuesFrom ex:Mother;
   owl:onProperty ex:femaleDescendant],
  [owl:intersectionOf (
    ex:Female
    [owl:someValuesFrom ex:Mother;
     owl:onProperty ex:isChildOfMother]])].
```

Consider the RDF triples presented in slide 26.

Inferred Triples

```
ex:Felicia a ex:Daughter.
ex:HumanFiona a ex:Daughter.
ex:OgreFiona a ex:Daughter.
ex:QueenLilian a ex:Daughter.
```

Inference from Complex Property Definition (5/5)

Definition of Complex Property:

```
ex:isChildOfMother a rdfs:Property,  
                   owl:InverseFunctionalProperty;  
                   owl:inverseOf ex:isMotherOf.
```

Inferred Triple

```
ex:HumanFiona owl:sameAs ex:OgreFiona.
```

Summary of OWL Concepts and Fragments (1/3)

Feature	Key Concepts	OWL Fragment
Class Definitions	<code>owl:intersectionOf</code> , <code>owl:unionOf</code> , <code>owl:allValuesFrom</code>	OWL Lite
Cardinality	<code>owl:minCardinality</code> , <code>owl:maxCardinality</code>	OWL DL
Property Characteristics	Functional, InverseFunctional, Symmetric, Asymmetric, Transitive	OWL Lite / OWL DL
Property Relationships	<code>rdfs:subPropertyOf</code> , <code>owl:inverseOf</code> , <code>owl:propertyDisjointWith</code>	OWL Lite / OWL DL

Summary of OWL Concepts and Fragments (2/3)

Feature	Key Concepts	OWL Fragment
Complex Class Inference	Inferences using <code>owl:someValuesFrom</code> , <code>owl:onClass</code> , <code>owl:qualifiedCardinality</code>	OWL Lite, OWL DL
Identity Inference	<code>owl:sameAs</code> , <code>owl:AllDifferent</code> Derived equivalences with <code>owl:InverseFunctionalProperty</code>	OWL Lite, OWL DL

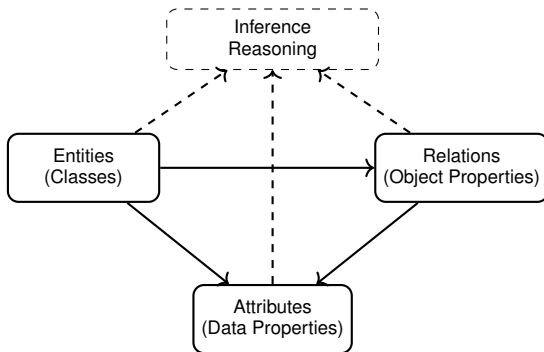
Summary of OWL Concepts and Fragments (3/3)

Feature	Key Concepts	OWL Fragment
Disjointness	<code>owl:disjointWith</code> , <code>owl:propertyDisjointWith</code>	OWL Lite, OWL DL
Negation	<code>owl:complementOf</code>	OWL DL
Additional Concepts	Meta-classes	OWL Full

Note: OWL Lite supports simpler constructs, OWL DL supports advanced features, and OWL Full provides the most flexibility but lacks computational guarantees.

Semantic Data Modeling in OWL

Goal of Semantic Modeling: Represent knowledge in a structured, machine-interpretable way.



Semantic Data Modeling enables: (1) structured domain knowledge; (2) automated reasoning; (3) interoperability and reuse across systems.

Next: Modeling basic concepts (entities, relationships, attributes) in OWL.

Semantic Data Modeling in OWL

Why Semantic Data Modeling?

- To represent domain knowledge in a structured way.
- To enable reasoning and inference over data.
- To ensure interoperability and reusability across systems.

What's Next?

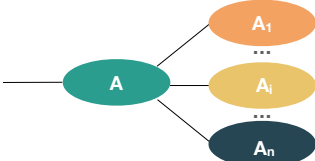
- Learn to model basic concepts (i.e., entities, relationships, and specializations) in OWL.

Attributes (1/2)^[1]

Attributes: properties of entities or relations between attributes

- **Simple:** the attribute takes a single and atomic value.
NameTeam is an atomic attribute that takes single values.
- **Composite:** the attribute is composed of several components.
Composition may form a hierarchy where some components are themselves composite.
Address is composed of the five attributes Street, Number, City, State, ZipCode, Country.
Name is composed of three attributes FirstName, MiddleName, LastName.
- **Multi-valued:** the attribute may have multiple values.
PhoneNumber of a person.
- **Optional:** the attribute may take no values.
PersonDegree of a person.

Attributes (2/2) [1]

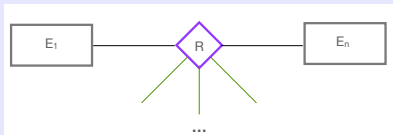
Type of Attribute	Notation
Simple	
Composite	
Multi-valued	
Optional	

Same notation to represent attributes in relationships and classes.

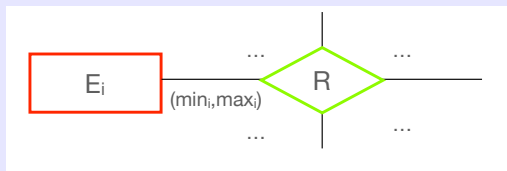
Relationships [1]

Relationship Types:

A diamond-shaped box displays a relationship, connecting to the participating entity types via straight lines from the diamond to the rectangular boxes.



Cardinality of a Relationship:

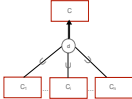
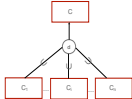
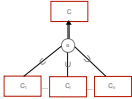
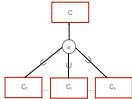


For all $e_i \in \mu(E_i)$, each entity from E_i is involved in min_i to max_i instances of $\mu(R)$:

$$min_i \leq |\{r \in \mu(R) : r = (\dots, e_i, \dots)\}| \leq max_i$$

default: (0, *), i.e., no restriction

Specializations- EER Notation [1]

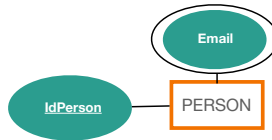
Specialization Type	Notation
Disjoint and Total Participation	
Disjoint and Partial Participation	
Overlap and Total Participation	
Overlap and Partial Participation	

From ER Modeling to OWL: Summary of Transformation

EER Concept	OWL Representation	Example
Attribute (simple, multivalued, optional)	<code>ex:A a owl:DatatypeProperty; rdfs:domain ex:E; rdfs:range D.</code>	<code>ex:Mail a owl:DatatypeProperty; rdfs:domain ex:Person; rdfs:range xsd:string.</code>
Binary Relationship (between two entities)	<code>ex:R a owl:ObjectProperty; rdfs:domain ex:C1; rdfs:range ex:C2.</code>	<code>ex:Affiliated a owl:ObjectProperty; rdfs:domain ex:Person; rdfs:range ex:University.</code>

Key idea: Attributes → `owl:DatatypeProperty` | Binary relationships → `owl:ObjectProperty`

Example: Attribute Transformation



```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix ex: <http://example.org/> .

```

```
ex:PERSON a owl:Class .
```

```

ex:IdPerson a          owl:DatatypeProperty ;
  rdfs:domain ex:PERSON ;
  rdfs:range  xsd:string .

```

```

ex:Mail a          owl:DatatypeProperty ;
  rdfs:domain ex:PERSON ;
  rdfs:range  xsd:string .

```

Example: Binary Relationship Transformation



```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .  
@prefix ex: <http://example.org/> .
```

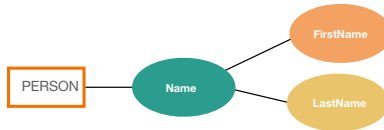
```
ex:PERSON      a owl:Class .  
ex:UNIVERSITY  a owl:Class .  
ex:Affiliated  a owl:ObjectProperty ;  
               rdfs:domain ex:PERSON ;  
               rdfs:range  ex:UNIVERSITY .
```

Modeling Composite Attributes in OWL

Composite Att	OWL Representation
Definition	A is composed of $A_1, \dots, A_n$, with datatypes $D_1, \dots, D_n$. A belongs to entity (or relationship) E .
Transformation	<pre>ex:A a owl:Class . ex:hasA a owl:ObjectProperty ; rdfs:domain ex:E ; rdfs:range ex:A . ex:A1 a owl:DatatypeProperty ; rdfs:domain ex:A ; rdfs:range D1 . : : ex:An a owl:DatatypeProperty ; rdfs:domain ex:A ; rdfs:range Dn .</pre>

Key idea: A composite attribute becomes a new OWL class. Its components become datatype properties with domain = that class. The entity links to this class through a new object property.

Example: Composite Attribute Transformation



```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix ex: <http://example.org/> .
  
```

```

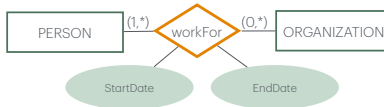
ex:NAME      a      owl:Class .
ex:PERSON    a      owl:Class .
ex:hasName   a      owl:ObjectProperty ;
              rdfs:domain ex:PERSON ;
              rdfs:range  ex:NAME .
ex:FirstName a      owl:DatatypeProperty ;
              rdfs:domain ex:NAME ;
              rdfs:range  xsd:string .
ex:LastName  a      owl:DatatypeProperty ;
              rdfs:domain ex:NAME ;
              rdfs:range  xsd:string .
  
```

Binary Relationships with Attributes in OWL

ER Concept	OWL Representation (Turtle)
Binary Relationship with Attributes	<pre>ex:R a owl:Class . ex:C1 a owl:Class . ex:C2 a owl:Class . ex:hasRC1 a owl:ObjectProperty ; rdfs:domain ex:R ; rdfs:range ex:C1 . ex:hasRC2 a owl:ObjectProperty ; rdfs:domain ex:R ; rdfs:range ex:C2 .</pre>

Key idea: A relationship with attributes becomes a new OWL class. Participants become object properties linking the class to each entity.

Example: Binary Relationship as a Class



```
ex:WORKFOR a owl:Class .
ex:hasWORKFORPerson a owl:ObjectProperty;
  rdfs:domain ex:WORKFOR;
  rdfs:range ex:PERSON.
ex:hasWORKFOROrganization a owl:ObjectProperty;
  rdfs:domain ex:WORKFOR ;
  rdfs:range ex:ORGANIZATION.
ex:StartDate a owl:DatatypeProperty;
  rdfs:domain ex:WORKFOR;
  rdfs:range xsd:date.
ex:EndDate a owl:DatatypeProperty;
  rdfs:domain ex:WORKFOR ;
  rdfs:range xsd:date.
```

Key idea: WORKFOR is modeled as a class; its participants are linked through object properties and attributes.

Defining Restrictions on `ex:PERSON`

```
ex:PERSON          a    owl:Class ;
                    owl:equivalentClass [
                        a owl:Restriction ;
                        owl:onProperty ex:hasWORKFOROrganization ;
                        owl:minCardinality "1"^^xsd:nonNegativeInteger;
                    ] .
```

Key Notes:

- `ex:PERSON`: Defined as equivalent to individuals with at least one `hasWORKFOROrganization` relationship.
- `owl:equivalentClass`: Ensures bidirectional inference:
 - Any individual with the relationship is classified as a `PERSON`.
 - All `PERSON` instances must satisfy the restriction.
- `owl:minCardinality 1`: Specifies that at least one such relationship is mandatory.

Complete Schema Overview

```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ex: <http://example.org/> .

ex:WORKFOR
    a      owl:Class.
ex:PERSON
    a      owl:Class ;
    owl:equivalentClass [
        a owl:Restriction ;
        owl:onProperty ex:hasWORKFOROrganization ;
        owl:minCardinality "1"^^xsd:nonNegativeInteger;
    ] .

ex:ORGANIZATION
    a      owl:Class.

ex:hasWORKFORPerson
    a      owl:ObjectProperty;
    rdfs:domain ex:WORKFOR;
    rdfs:range  ex:PERSON.

ex:hasWORKFOROrganization
    a      owl:ObjectProperty;
    rdfs:domain ex:WORKFOR;
    rdfs:range  ex:ORGANIZATION.

ex:StartDate
    a      owl:DatatypeProperty;
    rdfs:domain ex:WORKFOR;
    rdfs:range  xsd:date.

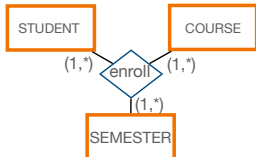
```


Modeling N-ary Relationships in OWL

N-ary Relationship	OWL Representation
Definition	A relationship R connects $C_1, C_2, \dots, C_n$. It is represented as a new class, with one object property per participant.
Transformation	<pre>ex:R a owl:Class. ex:hasRC1 a owl:ObjectProperty; rdfs:domain ex:R; rdfs:range ex:C1. : ex:hasRCn a owl:ObjectProperty; rdfs:domain ex:R; rdfs:range ex:Cn.</pre>

Notes: (1) Cardinality constraints are expressed using OWL restrictions on each property. (2) Attributes of the n-ary relationship become datatype properties whose domain is the class `ex:R`.

Modeling N-ary Relationships in OWL (1/2)



```
ex:enroll a owl:Class .  
ex:enrollSTUDENT a owl:ObjectProperty ;  
  rdfs:domain ex:enroll ;  
  rdfs:range ex:STUDENT .  
ex:enrollCOURSE a owl:ObjectProperty ;  
  rdfs:domain ex:enroll ;  
  rdfs:range ex:COURSE .  
ex:enrollSEMESTER a owl:ObjectProperty ;  
  rdfs:domain ex:enroll ;  
  rdfs:range ex:SEMESTER .
```

Key idea: An N-ary relationship becomes a new OWL class. Each participant becomes an object property whose domain is the new class.

Modeling Cardinality in N-ary Relationships (2/2)

```
# Exactly one student per enrollment
ex:enrollSTUDENT a owl:ObjectProperty ;
  rdfs:domain ex:enroll ;
  rdfs:range ex:STUDENT ;
  owl:qualifiedCardinality "1"8sd:nonNegativeInteger ;
  owl:onClass ex:STUDENT .

# At least one course per enrollment
ex:enrollCOURSE a owl:ObjectProperty ;
  rdfs:domain ex:enroll ;
  rdfs:range ex:COURSE ;
  owl:minQualifiedCardinality "1"8sd:nonNegativeInteger ;
  owl:onClass ex:COURSE .

# At least one semester per enrollment
ex:enrollSEMESTER a owl:ObjectProperty ;
  rdfs:domain ex:enroll ;
  rdfs:range ex:SEMESTER ;
  owl:minQualifiedCardinality "1"8sd:nonNegativeInteger ;
  owl:onClass ex:SEMESTER .
```

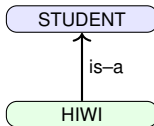
Transforming Taxonomies (1/2)

What is a Taxonomy? A taxonomy organizes classes in a hierarchy. If class C1 is more specific than class C2, then C1 is a *subclass* of C2.

OWL Representation (Turtle):

```
ex:C1 rdfs:subClassOf ex:C2 .
```

Visual Intuition:



Full Example (Turtle):

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .  
@prefix ex: <http://example.org/> .  
  
ex:STUDENT a owl:Class .  
ex:HIWI a owl:Class .  
ex:HIWI rdfs:subClassOf ex:STUDENT .
```

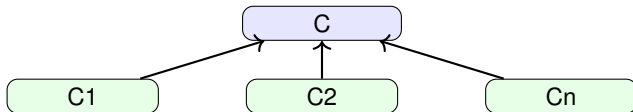
Transforming Taxonomies (2/2)

Definition: If a class C has several specializations ($C_1, C_2, \dots, C_n$), each one becomes a subclass of C in OWL.

OWL Representation:

```
ex:C1 rdfs:subClassOf ex:C .  
ex:C2 rdfs:subClassOf ex:C .  
...  
ex:Cn rdfs:subClassOf ex:C .
```

Visual Intuition:



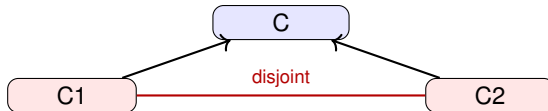
Transforming Disjoint Specializations

Definition: Disjoint specializations state that subclasses $C_1, C_2, \dots, C_n$ of a class C cannot share any instances.

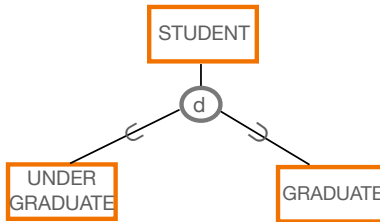
OWL Representation:

```
ex:C1 rdfs:subClassOf ex:C .  
ex:C2 rdfs:subClassOf ex:C .  
...  
ex:Cn rdfs:subClassOf ex:C .  
ex:C1 owl:disjointWith ex:C2 .  
ex:C1 owl:disjointWith ex:C3 .  
...  
# all  $C_i, C_j$  disjoint pairs ( $i \neq j$ )
```

Visual Intuition:



Example Disjoint Specializations



Transformation to OWL:

```
ex:STUDENT          a          owl:Class.  
ex:UNDERGRADUATE    a          owl:Class.  
ex:GRADUATE         a          owl:Class.  
ex:UNDERGRADUATE    rdfs:subClassOf ex:STUDENT.  
ex:GRADUATE         rdfs:subClassOf ex:STUDENT.  
  
ex:UNDERGRADUATE    owl:disjointWith ex:GRADUATE.
```

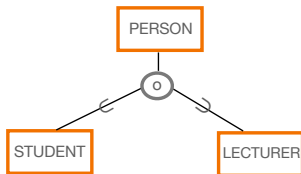
Transforming Overlapping Specializations

Definition: Overlapping specializations allow subclasses C1, C2, ..., Cn of class C to share common instances.

OWL Representation (Turtle):

```
ex:C1 rdfs:subClassOf ex:C .  
ex:C2 rdfs:subClassOf ex:C .  
...  
ex:Cn rdfs:subClassOf ex:C .
```

Example (OWL):



```
ex:PERSON a owl:Class .  
ex:STUDENT a owl:Class .  
ex:LECTURER a owl:Class .  
ex:STUDENT rdfs:subClassOf  
ex:PERSON .  
ex:LECTURER rdfs:subClassOf  
ex:PERSON .
```


Transforming Total and Partial Specializations

Definition:

- **Total specialization:** every instance of C must belong to at least one subclass $C_1, C_2, \dots, C_n$.
- **Partial specialization:** instances of C may exist that do not belong to any subclass.

Total Specialization (OWL):

```
ex:C owl:equivalentClass [  
  owl:unionOf (ex:C1 ex:C2  
  ex:Cn)  
] .
```

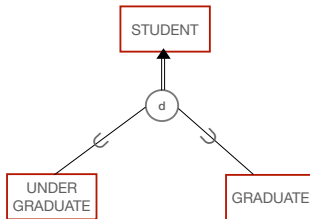
In total specialization, C is *fully partitioned* by its subclasses.

Partial Specialization (OWL):

```
ex:C1 rdfs:subClassOf ex:C .  
ex:C2 rdfs:subClassOf ex:C .  
# additional subclasses
```

In partial specialization, C may have *instances not covered by any subclass*.

Example: Total Disjoint Specialization



Transformation in OWL:

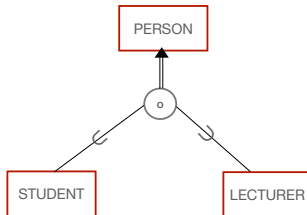
```
ex:STUDENT a          owl:Class.
ex:UNDERGRADUATE a    owl:Class.
ex:GRADUATE a         owl:Class.

ex:UNDERGRADUATE rdfs:subClassOf ex:STUDENT.
ex:GRADUATE rdfs:subClassOf ex:STUDENT.

ex:STUDENT owl:equivalentClass
  [owl:unionOf (ex:UNDERGRADUATE ex:GRADUATE)].

ex:UNDERGRADUATE owl:disjointWith ex:GRADUATE.
```

Example: Total Overlapping Specialization



Transformation into OWL

```
ex:PERSON      a      owl:Class.  
ex:STUDENT     a      owl:Class.  
ex:LECTURER    a      owl:Class.
```

```
ex:STUDENT     rdfs:subClassOf ex:PERSON.  
ex:LECTURER    rdfs:subClassOf ex:PERSON.
```

```
ex:PERSON owl:equivalentClass  
    [owl:unionOf (ex:STUDENT ex:LECTURER)].
```

Subsumption of Binary Relationship Types (1/2)

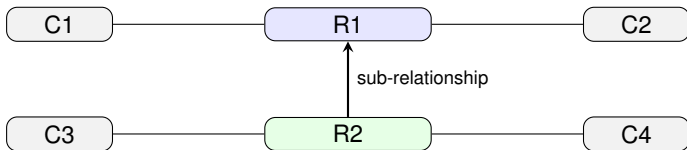
Definition:

- A relationship $R2$ is a **sub-relationship** of a relationship $R1$ if every instance of $R2$ is also an instance of $R1$.
- The relationships are defined over entity pairs:
 - $R1 \subseteq C1 \times C2$
 - $R2 \subseteq C3 \times C4$

Transformation Rule (OWL/RDFS):

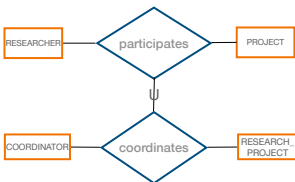
```
ex:R2 rdfs:subPropertyOf ex:R1 .
```

Visual Intuition:



Subsumption of Binary Relationship Types (2/2)

Representation in OWL and RDFS:



```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix ex: <http://example.org/> .

```

<code>ex:RESEARCHER</code>	<code>a</code>	<code>owl:Class.</code>
<code>ex:PROJECT</code>	<code>a</code>	<code>owl:Class.</code>
<code>ex:COORDINATOR</code>	<code>a</code>	<code>owl:Class.</code>
<code>ex:RESEARCH_PROJECT</code>	<code>a</code>	<code>owl:Class.</code>
<code>ex:participates</code>	<code>a</code>	<code>owl:ObjectProperty ;</code>
	<code>rdfs:domain</code>	<code>ex:RESEARCHER ;</code>
	<code>rdfs:range</code>	<code>ex:PROJECT .</code>
<code>ex:coordinates</code>	<code>a</code>	<code>owl:ObjectProperty ;</code>
	<code>rdfs:domain</code>	<code>ex:COORDINATOR ;</code>
	<code>rdfs:range</code>	<code>ex:RESEARCH_PROJECT .</code>
<code>ex:coordinates</code>	<code>rdfs:subPropertyOf</code>	<code>ex:participates .</code>

Subsumption of n-ary Relationships (1/3)

Definition: A relationship R2 is a **sub-relationship** of R1 if every tuple in R2 is also a tuple in R1.

Both relationships involve the same number of participating entity types:

R1 relates (C1, C2, Ci, ..., Cn)

R2 relates (D1, D2, Di, ..., Dn)

Visual Intuition:

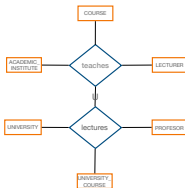


OWL Transformation:

```
ex:R2 rdfs:subClassOf ex:R1 .
```

Example: Subsumption of n -ary Relationships

Representation in OWL and RDFS:



```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix ex: <http://example.org/> .
```

```
ex:LECTURER          a          owl:Class.
ex:PROFESSOR          a          owl:Class.
ex:ACADEMIC_INSTITUTE a          owl:Class.
ex:UNIVERSITY         a          owl:Class.
```

```
ex:teaches            a          owl:Class.
ex:lectures           a          owl:Class ;
                        rdfs:subClassOf ex:teaches .
```

```
ex:teaches_LECTURER  a          owl:ObjectProperty ;
                        rdfs:domain ex:teaches ;
                        rdfs:range  ex:LECTURER .
```

```
ex:lectures_PROFESSOR a          owl:ObjectProperty ;
                        rdfs:domain ex:lectures ;
                        rdfs:range  ex:PROFESSOR .
```

Exercise

Participate **in the survey at Stud.IP.**

Examples of SPARQL Queries (1/2)

Query: How many owl:DatatypeProperty statements have each class?

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?class (COUNT(DISTINCT ?datatypeProperty) AS ?num)
WHERE {
  ?class a owl:Class .
  ?datatypeProperty a owl:DatatypeProperty.
  ?datatypeProperty rdfs:domain ?class.}
GROUP BY ?class
ORDER BY DESC (?num)
```

Examples of SPARQL Queries (2/2)

Query: Count the class(es) that have data types and object properties.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

```
SELECT (COUNT(DISTINCT ?class) AS ?num)
WHERE {
    ?class          a owl:Class .
    ?objectProperty a owl:ObjectProperty.
    ?objectProperty rdfs:domain ?class.
    ?datatypeProperty a owl:DatatypeProperty.
    ?datatypeProperty rdfs:domain ?class.}
```

Overview: Semantic Modeling in OWL

Concept	Representation in OWL
Attributes	Attributes are represented as <code>owl:DatatypeProperty</code> with a domain (class) and range (data type). Supports cardinality restrictions, functional properties, and typed literals.
Relationships	Binary relationships are represented as <code>owl:ObjectProperty</code> . Higher-arity relationships or relationships with attributes are modeled as RDF/OWL classes.
Classes	Entity types are represented as <code>owl:Class</code> . Support for equivalence (<code>owl:equivalentClass</code>), disjointness (<code>owl:disjointWith</code>), and hierarchical relationships (<code>rdfs:subClassOf</code>).
Taxonomies and Specializations	OWL supports rich taxonomies with total/partial and disjoint/overlapping specializations. Subclasses, equivalence, and intersections are defined using <code>rdfs:subClassOf</code> , <code>owl:equivalentClass</code> , and <code>owl:intersectionOf</code> .

Summary: Representing EER Schemas in OWL

Attributes

Simple Attributes →

`owl:DatatypeProperty`

- domain = class
- range = datatype

Composite Attributes →

`new owl:Class`

- sub-attributes become datatype properties

Relationships

Binary →

`owl:ObjectProperty`

N-ary → `new owl:Class` with

- one object property per participant
- relationship attributes as datatype properties

Classes

- Represented as `owl:Class`
- Hierarchies via `rdfs:subClassOf`
- Additional semantics: `owl:equivalentClass`, `owl:disjointWith`

Taxonomies

- Disjoint, overlapping, total, partial specializations
- Modeled with:
 - `rdfs:subClassOf`
 - `owl:unionOf`, `owl:intersectionOf`

OWL Features for Semantic Modeling

Properties

- **Datatype Properties**
(`owl:DatatypeProperty`)
→ attributes with literal values (strings, numbers, dates)
- **Object Properties**
(`owl:ObjectProperty`)
→ relationships between classes

Classes & Taxonomies

- **Hierarchy** →
`rdfs:subClassOf`
- **Disjointness** →
`owl:disjointWith`
- **Equivalence** →
`owl:equivalentClass`

Logical Constructors

- **Union** → `owl:unionOf`
- **Intersection** →
`owl:intersectionOf`
- **Complement** →
`owl:complementOf`

Cardinality Restrictions

- **Min / Max Cardinality** →
`owl:minCardinality`,
`owl:maxCardinality`
- **Qualified Cardinality** → restrict both number *and* type of values
(`owl:minQualifiedCardinality`)

References I



R. Elmasri and S. B. Navathe.

Fundamentals of Database Systems, 2nd Edition.

Benjamin/Cummings, 1994.